AI-NATIVE GROCERY MERCHANT

Document 07 — Data Model Specification

Development-only specification • Locked MVP • Persistence model, entities, relationships, state and integrity rules

> **Data principle: Authoritative commerce state lives in the backend. AI reasoning is stored as structured decision metadata and evidence references, not as authoritative financial state.**

Document status: LOCKED DEVELOPMENT SPECIFICATION

Recommended persistence: Relational database with explicit foreign keys, constraints, timestamps and append-oriented audit records.

1. Purpose and Scope

This document defines the persistent data model for the locked AI-Native Grocery Merchant MVP. It specifies core entities, fields, relationships, state transitions, monetary representation, integrity constraints, audit records and data ownership.

The model is intentionally designed for a modular monolith and a small controlled MVP. It should support the complete path from user grocery intent through optimization, user basket selection, policy authorization, Razorpay payment and final audit trace without introducing unnecessary platform complexity.

1.1 In scope

• User/session context required by the MVP.

• Agent runs and shopping intents.

• Mandates and mandate constraints.

• Requirements and candidate products.

• Catalog products, packs and stock.

• Quality/research evidence.

• Optimization runs and basket candidates.

• Voucher and loyalty incentive state.

• Basket and basket-item state.

• Policy decisions.

• Orders and payment state.

• Audit events.

1.2 Out of scope

• Multi-merchant marketplace data model.

• Production-scale recommendation infrastructure.

• Full loyalty platform.

• General external coupon marketplace.

• Banking/payout/refund ledger.

• Full ACP/AP2/UAP protocol data model.

2. Data Ownership Model

Domain	Authoritative owner	AI role
User goal	User/application	Interpret
Mandate	Backend authorization store	Read only
Product identity	Catalog	Compare
Price	Catalog/commerce	Read only
Stock	Catalog/inventory	Read only
Quality evidence	Approved research store	Interpret
Incentive rules	Incentive service	Evaluate value
Basket total	Cart/commerce	Explain
Policy decision	Policy engine	Request/react
Razorpay order/payment	Payment integration	Never authoritative
Audit history	Audit store	Provide event metadata

Rule: No AI-generated field may become authoritative merely because it is persisted. Authoritative fields must originate from their owning backend domain.

3. Entity Relationship Overview

```
USER / SESSION
■
■■■ SHOPPING_INTENT
■ ■
■ ■■■ REQUIREMENT
■ ■
■ ■■■ OPTIMIZATION_RUN
■ ■
■ ■■■ CANDIDATES
■ ■■■ BASKET (BEST_VALUE)
■ ■■■ BASKET (BEST_QUALITY)
■
■■■ MANDATE
■
■■■ POLICY_DECISION
■
■■■ ORDER
■
■■■ PAYMENT

PRODUCT
■■■ PRODUCT_PACK
■■■ STOCK
■■■ QUALITY_EVIDENCE

INCENTIVE
■■■ VOUCHER
■■■ LOYALTY_REWARD

AUDIT_EVENT
■■■ references intent / mandate / optimization / basket /
policy / order / payment
```

Foreign-key relationships should be explicit wherever the underlying domain object exists. JSON metadata may be used for flexible AI traces, but it must not replace relational ownership for critical commerce state.

4. Identifier and Naming Standards

Object	Identifier recommendation
User	user_id
Agent run	agent_run_id
Mandate	mandate_id
Shopping intent	intent_id
Requirement	requirement_id
Product	product_id
SKU/pack	sku_id
Evidence	evidence_id
Optimization run	optimization_run_id
Basket	basket_id
Basket item	basket_item_id
Incentive	incentive_id
Policy decision	policy_decision_id
Order	order_id
Payment	payment_id

Object	Identifier recommendation
Audit event	audit_event_id

UUIDs are recommended for externally exposed identifiers. Internal database keys may use UUIDs directly for simplicity in the MVP.

5. Common Field Conventions

5.1 Timestamps

Persist timestamps in UTC. API responses may render them in the user's local timezone. Use created_at and updated_at consistently. State-transition events should additionally store the event timestamp when useful.

5.2 Money

All monetary values must be stored as integer minor currency units. For INR, ■1 = 100 paise.

```
gross_amount = 58600
discount_amount = 0
final_payable = 58600

# UI representation:
■586
```

Do not store financial values as floating-point numbers.

5.3 Status fields

Statuses should use controlled enumerations rather than arbitrary free text. Invalid state transitions must be rejected by application logic and, where practical, database constraints.

6. User and Session Model

6.1 User

```
user
-----
user_id UUID PK
display_name VARCHAR
currency VARCHAR(3) DEFAULT 'INR'
created_at TIMESTAMP
updated_at TIMESTAMP
```

The MVP requires only the minimum user identity/context needed to associate mandates, shopping sessions, baskets and payments. Do not add a full customer profile unless required by implementation.

6.2 Shopping session

```
shopping_session
----------------
session_id UUID PK
user_id UUID FK
status ENUM
started_at TIMESTAMP
ended_at TIMESTAMP NULL
created_at TIMESTAMP
```

A session groups one shopping workflow. It should not be treated as a payment authorization.

7. Agent Run Model

7.1 Agent run

```
agent_run
---------
agent_run_id UUID PK
session_id UUID FK
agent_version VARCHAR
status ENUM
current_stage VARCHAR
```

```
started_at TIMESTAMP
completed_at TIMESTAMP NULL
failure_code VARCHAR NULL
created_at TIMESTAMP
```

Agent runs provide operational traceability without storing hidden chain-of-thought.

7.2 Agent event

```
agent_event
-----------
agent_event_id UUID PK
agent_run_id UUID FK
event_type VARCHAR
tool_name VARCHAR NULL
decision_code VARCHAR NULL
input_ref VARCHAR NULL
output_ref VARCHAR NULL
metadata_json JSON
created_at TIMESTAMP
```

metadata_json may contain compact decision facts, tool identifiers and evidence references. Secrets and raw payment credentials must never be stored.

8. Mandate Model

8.1 Mandate

```
mandate
-------
mandate_id UUID PK
user_id UUID FK
agent_id VARCHAR
max_spend_minor BIGINT
currency VARCHAR(3)
max_per_item_minor BIGINT NULL
purpose TEXT
valid_until TIMESTAMP
status ENUM
created_at TIMESTAMP
updated_at TIMESTAMP
```

8.2 Mandate category

```
mandate_category
----------------
mandate_id UUID FK
category VARCHAR
PRIMARY KEY (mandate_id, category)
```

Allowed categories should be normalized rather than stored as an opaque JSON array when they participate directly in authorization queries.

8.3 Mandate invariants

• max_spend_minor >= 0.

• max_per_item_minor >= 0 when present.

• currency must be supported by the application.

• valid_until must be later than creation time for an active mandate.

• Only authorized backend flows can change mandate state.

• The agent has no write access to mandate authorization fields.

9. Shopping Intent and Requirement Model

9.1 Shopping intent

```
shopping_intent
---------------
intent_id UUID PK
```

```
session_id UUID FK
mandate_id UUID FK
goal_text TEXT
category VARCHAR
budget_minor BIGINT NULL
quality_preference VARCHAR NULL
status ENUM
assumptions_json JSON
created_at TIMESTAMP
updated_at TIMESTAMP
```

budget_minor may be stored as the user's stated budget, but authorization remains governed by the mandate and policy engine.

9.2 Requirement

```
requirement
-----------
requirement_id UUID PK
intent_id UUID FK
item_name VARCHAR
target_quantity DECIMAL
unit VARCHAR
minimum_quality VARCHAR NULL
constraints_json JSON
confidence DECIMAL NULL
status ENUM
created_at TIMESTAMP
updated_at TIMESTAMP
```

A requirement represents what the user needs, not which product will satisfy it.

10. Catalog Model

10.1 Product

```
product
-------
product_id UUID PK
merchant_id UUID NULL
name VARCHAR
description TEXT
category VARCHAR
brand VARCHAR NULL
status ENUM
created_at TIMESTAMP
updated_at TIMESTAMP
```

The MVP may use a single merchant. merchant_id can remain optional or be retained for future-safe internal organization without implementing a marketplace.

10.2 SKU / pack

```
sku
---
sku_id UUID PK
product_id UUID FK
sku_code VARCHAR UNIQUE
pack_quantity DECIMAL
pack_unit VARCHAR
price_minor BIGINT
currency VARCHAR(3)
status ENUM
created_at TIMESTAMP
updated_at TIMESTAMP
```

10.3 Stock

```
stock
-----
sku_id UUID PK/FK
available_quantity DECIMAL
updated_at TIMESTAMP
```

Price and stock must be authoritative backend data. The agent may cache or reason over snapshots, but checkout uses current state.

11. Quality and Evidence Model

11.1 Quality evidence

```
quality_evidence
----------------
evidence_id UUID PK
product_id UUID FK
sku_id UUID NULL FK
source_type VARCHAR
source_reference VARCHAR
summary TEXT
quality_signal VARCHAR
confidence DECIMAL
captured_at TIMESTAMP
expires_at TIMESTAMP NULL
created_at TIMESTAMP
```

11.2 Evidence rules

• Store evidence summaries, not unsupported model claims.

• Keep source_reference for traceability where available.

• Confidence should represent evidence quality, not certainty invented by the model.

• Expired evidence may be excluded from new optimization runs.

• Material product decisions should retain evidence_id references.

For the locked MVP, seeded/controlled evidence is preferred over an unrestricted web research store.

12. Optimization Model

12.1 Optimization run

```
optimization_run
----------------
optimization_run_id UUID PK
intent_id UUID FK
mandate_id UUID FK
agent_run_id UUID FK
status ENUM
candidate_count INTEGER
started_at TIMESTAMP
completed_at TIMESTAMP NULL
created_at TIMESTAMP
```

12.2 Optimization candidate

```
optimization_candidate
----------------------
candidate_id UUID PK
optimization_run_id UUID FK
requirement_id UUID FK
candidate_data_json JSON
gross_amount_minor BIGINT
quality_score DECIMAL NULL
covered_quantity DECIMAL
status ENUM
created_at TIMESTAMP
```

Candidate data may store the bounded combination proposed by the optimizer. Authoritative price/stock should still be resolved from catalog/cart services before authorization.

13. Basket Model

13.1 Basket

```
basket
------
basket_id UUID PK
optimization_run_id UUID FK
basket_type ENUM
status ENUM
gross_amount_minor BIGINT
discount_amount_minor BIGINT
final_payable_minor BIGINT
currency VARCHAR(3)
quality_summary TEXT
recommendation_reason TEXT NULL
created_at TIMESTAMP
updated_at TIMESTAMP
```

basket_type is expected to include BEST_VALUE and BEST_QUALITY for the locked MVP.

13.2 Basket item

```
basket_item
-----------
basket_item_id UUID PK
basket_id UUID FK
requirement_id UUID FK
sku_id UUID FK
quantity DECIMAL
unit_price_minor BIGINT
line_amount_minor BIGINT
quality_level VARCHAR NULL
evidence_refs_json JSON
created_at TIMESTAMP
updated_at TIMESTAMP
```

unit_price_minor and line_amount_minor can be snapshots for audit/explanation, but the current authoritative quote must be recalculated before payment.

14. Incentive Model

14.1 Incentive

```
incentive
---------
incentive_id UUID PK
type ENUM
name VARCHAR
description TEXT
status ENUM
valid_from TIMESTAMP NULL
valid_until TIMESTAMP NULL
rules_json JSON
created_at TIMESTAMP
updated_at TIMESTAMP
```

type should support at least VOUCHER and LOYALTY_REWARD.

14.2 Incentive evaluation

```
incentive_evaluation
--------------------
evaluation_id UUID PK
optimization_run_id UUID FK
basket_id UUID NULL FK
incentive_id UUID FK
decision ENUM
current_saving_minor BIGINT
future_value_minor BIGINT NULL
reason TEXT
eligibility_snapshot JSON
created_at TIMESTAMP
```

decision should support USE_NOW, SAVE_FOR_LATER and DO_NOT_USE.

14.3 Important rule

The database must distinguish the incentive's actual deterministic benefit from the AI's decision reason. The AI does not write authoritative discount amounts.

15. User Selection Model

15.1 Basket selection

```
basket_selection
----------------
selection_id UUID PK
session_id UUID FK
basket_id UUID FK
selection_source ENUM
selected_at TIMESTAMP
superseded_at TIMESTAMP NULL
created_at TIMESTAMP
```

The locked flow expects selection_source = USER for the final basket choice.

15.2 Selection invariant

A basket selection does not authorize payment. It only identifies the user's chosen proposal. The backend must re-quote and re-run policy before payment.

16. Policy Decision Model

16.1 Policy decision

```
policy_decision
---------------
policy_decision_id UUID PK
mandate_id UUID FK
basket_id UUID FK
decision ENUM
reason_code VARCHAR
gross_amount_minor BIGINT
discount_amount_minor BIGINT
final_payable_minor BIGINT
max_spend_minor BIGINT
policy_version VARCHAR
evaluated_at TIMESTAMP
request_id VARCHAR
created_at TIMESTAMP
```

16.2 Policy checks

The policy decision must represent the result of the ordered checks defined in Document 06: mandate validity, category, stock, maximum per item, incentive validity, final amount calculation and maximum spend.

16.3 Decision invariants

• ALLOW requires all mandatory checks to pass.

• DENY always has a reason_code.

• final_payable_minor is authoritative for max-spend evaluation.

• policy_version is required.

• A policy decision cannot be modified by the AI.

17. Order Model

17.1 Order

```
order
-----
order_id UUID PK
user_id UUID FK
session_id UUID FK
mandate_id UUID FK
basket_id UUID FK
policy_decision_id UUID FK
status ENUM
gross_amount_minor BIGINT
discount_amount_minor BIGINT
```

```
final_payable_minor BIGINT
currency VARCHAR(3)
razorpay_order_id VARCHAR NULL UNIQUE
created_at TIMESTAMP
updated_at TIMESTAMP
```

An order must reference the exact basket and policy decision that authorized it. A payment order must never exist without the corresponding successful authorization path.

17.2 Order states

```
CREATED
↓
PAYMENT_PENDING
■■■ PAYMENT_VERIFIED
■■■ PAYMENT_FAILED
■■■ PAYMENT_EXPIRED / CANCELLED
↓
COMPLETED (if application defines a separate final state)
```

18. Payment Model

18.1 Payment

```
payment
-------
payment_id UUID PK
order_id UUID FK
razorpay_payment_id VARCHAR NULL UNIQUE
status ENUM
amount_minor BIGINT
currency VARCHAR(3)
method VARCHAR NULL
verified_at TIMESTAMP NULL
failure_code VARCHAR NULL
metadata_json JSON
created_at TIMESTAMP
updated_at TIMESTAMP
```

18.2 Payment integrity

• Payment amount must match the authorized order amount.

• Payment success must be based on backend verification/webhook state.

• The agent cannot set payment status to successful.

• Duplicate webhook events must not create duplicate payment records.

• Secrets and sensitive credentials must not be stored in ordinary metadata.

19. Audit Event Model

19.1 Audit event

```
audit_event
-----------
audit_event_id UUID PK
event_type VARCHAR
user_id UUID NULL FK
session_id UUID NULL FK
agent_run_id UUID NULL FK
mandate_id UUID NULL FK
optimization_run_id UUID NULL FK
basket_id UUID NULL FK
policy_decision_id UUID NULL FK
order_id UUID NULL FK
payment_id UUID NULL FK
event_data_json JSON
occurred_at TIMESTAMP
created_at TIMESTAMP
```

19.2 Event examples

Event type	Minimum useful data
INTENT_CREATED	Goal, category, mandate reference
REQUIREMENT_EXTRACTED	Requirement summary and assumptions
PRODUCT_SELECTED	SKU, requirement, evidence refs
INCENTIVE_DECISION	Incentive, decision, current saving, reason
BASKET_CREATED	Basket type and authoritative quote snapshot
BASKET_RECOMMENDED	Basket id, reason, confidence
USER_BASKET_SELECTED	Selected basket and timestamp
POLICY_DECISION	ALLOW/DENY, reason, amounts, policy version
POLICY_RECOVERY	Denial and resulting re-optimization
RAZORPAY_ORDER_CREATED	Order and Razorpay order reference
PAYMENT_VERIFIED	Payment reference, amount and verified state

20. State Ownership and Mutation Rules

Entity	AI can create proposal data	AI can mutate authoritative state
Shopping intent	Yes, through controlled backend flow	No
Requirements	Yes	No
Mandate	No	No
Catalog product	No	No
Quality evidence	Reference only	No
Optimization run	Yes, proposal metadata	No authoritative price/stock
Basket	Yes, proposed basket	No authoritative final amount
Incentive evaluation	Yes, decision rationale	No rule/eligibility mutation
Policy decision	Request only	No
Order	No direct creation	No
Payment	No	No
Audit	Event request/metadata only	No historical mutation

All writes from the agent should pass through typed application services that enforce the same domain boundaries described in the architecture.

21. Referential Integrity

• Every basket belongs to exactly one optimization run.

• Every basket item belongs to exactly one basket.

• Every basket item references an existing SKU and requirement.

• Every policy decision references a mandate and basket.

• Every order references a policy decision that authorized it.

• Every payment references an order.

• Audit references may be nullable because not every event occurs at every stage.

• Deleting historical commerce objects should generally be prohibited; use status/retention policies instead.

22. Uniqueness and Constraint Recommendations

Constraint	Recommendation
sku_code	UNIQUE
razorpay_order_id	UNIQUE when present
razorpay_payment_id	UNIQUE when present
mandate category	UNIQUE per mandate/category
Active basket selection	At most one active selection per session
Policy request	Unique request/idempotency key within authorization scope
Audit event id	UNIQUE
Money values	CHECK >= 0 where applicable
Quantity	CHECK > 0 for purchased basket items

23. Transaction Boundaries

23.1 Basket selection + quote

User selection and the initial quote may be separate operations. The payment path must always obtain a fresh authoritative quote before policy authorization.

23.2 Policy + order creation

Where practical, authorization and order creation should be protected against races using a short-lived authorization token, transaction, or equivalent server-side mechanism. If the implementation cannot guarantee atomicity, it must revalidate before order creation and fail closed on mismatch.

23.3 Webhook processing

Webhook handling must be idempotent. The database should record the external event identifier or an equivalent deduplication key where available.

24. Snapshot vs Live Data

Data	Snapshot allowed?	Payment path uses
Product name	Yes	Current or validated snapshot
Quality evidence	Yes	Relevant current evidence
Unit price	Yes for audit	Fresh authoritative price
Stock	Yes for planning	Fresh authoritative stock
Voucher rule	Yes for planning	Fresh validation
Basket total	Yes for display/audit	Fresh authoritative quote
Policy result	Yes	Current valid authorization

Planning snapshots are useful for reproducibility, but they never override live authoritative state at authorization.

25. Example End-to-End Record Chain

```
user
user-001
■
■■■ mandate-001 (max ■1000)
■
■■■ session-001
■
■■■ intent-001
■ ■■■ req-pasta
■ ■■■ req-sauce
■
```

```
■■■ optimization-001
■■■ candidate-001...
■■■ basket-value-001
■■■ basket-quality-001
■■■ recommendation metadata
■
■■■ selection-001
■
■■■ policy-001 = ALLOW
■
■■■ order-001
■■■ payment-001

audit_event records connect each material transition.
```

This chain should allow a developer or evaluator to start from a payment and trace backward to the user's goal, mandate, selected basket, policy authorization and optimization decisions.

26. Data Retention and Privacy

The MVP should retain enough information to reproduce and explain the transaction path while avoiding unnecessary personal data.

• Store only user information needed by the application.

• Do not store payment credentials.

• Avoid storing raw sensitive payment payloads when references are sufficient.

• Keep audit records long enough for development/debugging and evaluation requirements.

• If records are removed under a retention policy, preserve non-sensitive aggregate integrity where required.

• Do not use persisted AI traces as a substitute for authorization records.

27. Migration and Seed Data Requirements

The development database should include deterministic seed data sufficient to demonstrate the locked optimization cases.

27.1 Minimum seed data

• At least one grocery merchant/catalog context.

• Multiple products with different quality evidence.

• Multiple pack sizes for quantity optimization.

• Products with different prices and quality trade-offs.

• At least one valid voucher.

• At least one voucher suitable for SAVE_FOR_LATER.

• At least one loyalty reward scenario.

• Stock states sufficient to trigger replacement/recovery.

• Mandates with different spend limits.

• Products that exercise Best Value vs Best Quality.

27.2 Required demonstration case

The seeded catalog should support the six-egg example where a poor-quality six-pack is inferior to three good-quality two-packs at the same total price. This verifies that the optimizer is not merely selecting the cheapest SKU.

28. Data Validation Rules

• IDs must be valid UUIDs where UUIDs are used.

• Quantities must be positive for basket items.

• Money values must be integer minor units.

• Final payable cannot be negative.

• Discount cannot exceed gross amount unless a future business rule explicitly supports credits/refunds; the locked MVP should reject such states.

• Basket type must be BEST_VALUE or BEST_QUALITY for the locked MVP output.

• Incentive decision must be USE_NOW, SAVE_FOR_LATER or DO_NOT_USE.

• Policy decision must be ALLOW or DENY.

• DENY requires a reason code.

• ALLOW requires successful mandatory checks.

• Order creation requires a valid policy authorization reference.

29. Definition of Done

• All core entities are implemented with stable identifiers.

• Mandates and authorization constraints are persisted server-side.

• Shopping intents and requirements are persisted.

• Catalog products, SKUs and stock are authoritative backend records.

• Quality evidence is referenceable from optimization decisions.

• Optimization runs and candidates are traceable.

• Best Value and Best Quality baskets are persisted separately.

• Basket items preserve requirement and evidence references.

• Voucher and loyalty evaluation decisions are persisted.

• User basket selection is persisted and remains separate from authorization.

• Policy decisions store decision, reason, amounts and policy version.

• Orders reference the exact policy decision used for authorization.

• Payments reference orders and support idempotent state updates.

• Audit events connect the complete decision path.

• Money is stored as integer minor units.

• Referential and uniqueness constraints are enforced.

• Seed data supports the locked demo scenarios.

• No data model for out-of-scope functionality is introduced.

30. Related Development Documents

Document	Relationship
01 — Development Master Spec	Locked product and engineering baseline
02 — Functional Requirements	Functional behavior represented by these entities
03 — System Architecture	Component ownership and trust boundaries
04 — AI Agent Specification	Agent state, tool interactions and decision metadata
05 — Optimization Decision Specification	Optimization inputs/outputs persisted here
06 — Policy & Mandate Specification	Authorization fields and policy decisions
08 — API Contract Specification	External/service payloads mapped to this model

Document	Relationship
09 — Razorpay Integration Specification	Order/payment persistence and external references
10 — Testing & Acceptance Specification	Data integrity and workflow validation